



**School of Industrial and Information Engineering**  
MSc in Mechanical Engineering (FA4 – Mechatronics and Robotics)

**Machine Learning for Mechanical Systems**

**AA 2025-2026**

**Projects report**

**Authors list**

Yigiter Tuncer

**Professor Roveda Loris**

## Index

|                                                                                         |           |
|-----------------------------------------------------------------------------------------|-----------|
| <b>1. MATLAB Project Parcel Singulation with AMS using Reinforcement Learning .....</b> | <b>3</b>  |
| <b>1.1 Introduction .....</b>                                                           | <b>3</b>  |
| 1.1.1 Motivation and Industrial Context .....                                           | 3         |
| 1.1.2 Functional Requirements & Verification Matrix .....                               | 3         |
| 1.1.3 Scope and Objectives.....                                                         | 4         |
| 1.1.4 Why RL versus Rule-based Systems? .....                                           | 4         |
| <b>1.2 System Description .....</b>                                                     | <b>4</b>  |
| 1.2.1 Physical Environment and Parcel Spawn Mechanism .....                             | 4         |
| 1.2.2 Physics Engine & Kinematics .....                                                 | 5         |
| 1.2.3 Coordinate System & Conventions .....                                             | 5         |
| 1.3.2 Action Space Discount Factor and Episode Termination .....                        | 5         |
| <b>1.4 Algorithm: PPO - Proximal Policy Optimization .....</b>                          | <b>6</b>  |
| 1.4.1 Overview of Policy Gradient Methods and PPO Core Idea .....                       | 6         |
| 1.4.2 Generalized Advantage Estimate (GAE) .....                                        | 6         |
| 1.4.3 Neural Network Architecture and Hyperparameter Sensitivity.....                   | 6         |
| <b>1.5 Reward Function Design .....</b>                                                 | <b>6</b>  |
| 1.5.1 Reward Shaping Concept .....                                                      | 6         |
| 1.5.2 Iterative Design Process and Observations .....                                   | 7         |
| 1.5.3 Final Reward Architecture.....                                                    | 7         |
| <b>1.6 Results and Ablation Study.....</b>                                              | <b>8</b>  |
| 1.6.1 Training Convergence & Before vs. After Training .....                            | 8         |
| 1.6.2 Speed Analysis & Performance Metrics.....                                         | 10        |
| 1.6.3 Ablation Study & Video Comparison .....                                           | 10        |
| 1.6.4 Qualitative Analysis & Bottlenecks.....                                           | 10        |
| <b>1.7 Conclusion .....</b>                                                             | <b>11</b> |
| 1.7.1 Summary of Contributions .....                                                    | 11        |
| 1.7.2 Fundamental Engineering Outcomes.....                                             | 11        |
| 1.7.3 Limitations.....                                                                  | 12        |
| 1.7.4 Future Work .....                                                                 | 12        |

# 1. MATLAB Project Parcel Singulation with AMS using Reinforcement Learning

## 1.1 Introduction

### 1.1.1 Motivation and Industrial Context

The exponential growth of e-commerce creates major bottlenecks in modern logistics. Traditional sorters (cross-belt, pop-up rollers) lack flexibility for variable parcel sizes and chaotic arrival rates. Parcel singulation – arranging scattered parcels into a single, evenly spaced line – is a critical preprocessing step for scanning, weighing and routing. Automating it with intelligent, decentralised systems is essential to maximise throughput and reduce manual intervention.

The Active-Matrix Separator (AMS) represents a shift from unidirectional conveyors to fully distributed, omnidirectional material handling. In this project the AMS is modelled as a  $5 \times 5$  grid of independent motor cells, each  $0.2 \text{ m} \times 0.2 \text{ m}$ , for a total active area of  $1.0 \text{ m}^2$ . Unlike standard conveyors, which apply a uniform force, the AMS can impose distinct local velocity and rotation vectors on each parcel simultaneously, making it an ideal environment for Reinforcement Learning (RL) agents managing multi-parcel traffic



Figure 1: AMS Conveyor Machine

### 1.1.2 Functional Requirements & Verification Matrix

To ensure the system meets industrial standards and to provide a structured evaluation framework, the project is limited to four core Functional Requirements (FRs). These requirements are validated through specific simulation metrics summarized in the validation table below.

| FR ID | Requirement Definition                                                                                           | Means of Verification                                                                                    |
|-------|------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| FR1   | <b>Cargo Singulation:</b> Cargoes must be combined into a single, continuous row along the Y-axis.               | This is assessed by measuring the cumulative accuracy (%) at the AMSVisualizer's evaluation line at 1 m. |
| FR2   | <b>Minimum Gap:</b> A minimum longitudinal gap of 15 cm (0.15 m) must be maintained between consecutive cargoes. | The lastGap metric is used to derive this value during simulation studies                                |

|     |                                                                                                                      |                                                                                                           |
|-----|----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
|     |                                                                                                                      | (Operational target: average of 0.89 m - 1.30 m).                                                         |
| FR3 | <b>Collision Avoidance:</b> Physical intersections between bounding boxes must be strictly prevented.                | Each episode is monitored via the collision penalty counter in the RL environment.                        |
| FR4 | <b>Centerline Evacuation:</b> Cargo should exit the AMS matrix from a location close to the longitudinal centerline. | The lateral deviation from the $X = 0.5$ m axis at the starting line is measured by statistical analysis. |

### 1.1.3 Scope and Objectives

This report details the end-to-end development, training, and evaluation processes of a Proximal Policy Optimization (PPO) agent tasked with monitoring AMS. The project scope includes the mathematical formulation of the Markov Decision Process (MDP), iterative design of the reward function (reward shaping) to overcome complex behavioral edge cases such as the "Catch-22" speed paradox and pseudo-collisions, and a comprehensive ablation study of system bottlenecks. So, it can be based on three main axes:

1. **Mathematical Modeling:** Formulating the environment as a Markov Decision Process (MDP) and mapping physical boundaries to a continuous action space.
2. **Reward Ablation:** Conducting a component effect analysis (ablation study) across 10 different reward architectures (iterations) to overcome behavioral bottlenecks such as the "Catch-22" speed paradox and X-axis pseudo-collisions.
3. **Hyperparameter Sensitivity Analysis:** Evaluating the learning dynamics of the agent. In this context, controlled failure scenarios were examined where reducing the PPO entropy value and increasing the network update frequency (Batch/Epoch ratios) leads to policy collapse to local optimum and physical bottlenecks.

The primary objective is to prove that the agent achieves 100% accuracy in both isolated wave-based and time-based spawn scenarios, while also systematically identifying the theoretical throughput limits and deadlock points of the AMS hardware.

### 1.1.4 Why RL versus Rule-based Systems?

In a system like AMS, with a 50-dimensional continuous action space and 5 times 5's of operation, writing traditional IF-THEN rules (rule-based logic) for each motor is nearly impossible. Because box sizes, arrival speeds, and positions on the conveyor belt are constantly changing, a rule-based system quickly locks into a "combinatorial explosion." Reinforcement Learning (RL), instead of manually coding the system, tells the agent "What to do" (via a reward function) and allows it to figure out "how to do it" on its own. This enables the model to make flexible decisions by generalizing, even in scenarios the programmer cannot foresee.

## 1.2 System Description

### 1.2.1 Physical Environment and Parcel Spawn Mechanism

The simulation reproduces an AMS made of 25 independent motorised cells ( $5 \times 5$ , each  $0.2 \times 0.2$  m), giving a  $1.0 \text{ m} \times 1.0 \text{ m}$  active area. Immediately downstream ( $Y > 1.0 \text{ m}$ ) a passive outfeed conveyor

moves at a constant 1.9 m/s. The integration step is  $\Delta t = 0.01$  s (100 Hz). The environment supports up to 10 parcels simultaneously, with sizes drawn uniformly in [0.05, 0.40] m.

Two spawn mechanisms are used. In wave-based scenarios two parcels are injected initially and new ones are added once previous ones are evacuated (with an 8.0 s timeout safeguard). In time-based stress tests two new parcels are injected every 0.75 s regardless of the matrix state, probing the system limits and exposing throughput bottlenecks.

### 1.2.2 Physics Engine & Kinematics

At each  $\Delta t$  the centre of every parcel is updated from its commanded forward speed  $v$  and orientation  $\theta$  as:

$$x_{(t+1)} = x_t + v * \Delta t * \sin(\theta)$$

$$y_{(next)} = y_{(current)} + v * \Delta t * \cos(\theta)$$

A collision-resolution mechanism separates overlapping bodies, leaving a tolerance distance  $toll\_contatto = 0.005$  m. To avoid the agent perceiving the resolved contacts as continuous collisions, the penalty threshold in the RL layer is synchronised at  $safe\_dist + 0.006$  m.

A dimensional mismatch present in the raw environment data was corrected: on the AMS the forward velocity  $vy\_boxes$  is expressed in m/step, while on the outfeed belt it is already in m/s. All velocities are converted to true metric units ( $vy\_ms = vy\_boxes / \Delta t$ ) before being passed to the agent, removing a systematic perception error.

### 1.2.3 Coordinate System & Conventions

A 2D Cartesian frame is used with the origin at the lower-left corner. The X-axis (lateral, 0–1 m) carries the centerline target  $X = 0.5$  m; the Y-axis (longitudinal, flow direction) covers the 0–1 m controlled region and  $Y > 1.0$  m for the passive outfeed. The 15 cm gap defined by FR2 is measured as the absolute distance between the trailing edge of the front parcel and the leading edge of the rear parcel, not between centroids.

## 1.3 MDP - Reinforcement Learning Formulation

### 1.3.1 Markov Decision Process (MDP) Framework and State Space

The singulation problem is formulated as a Markov Decision Process defined by states (S), actions (A), transition probabilities (P), reward (R) and discount factor ( $\gamma$ ). To preserve the Markov property under partial observability, the previous action vector is concatenated into the current observation, providing kinematic memory to the policy.

The observation is a fixed-length  $100 \times 1$  column vector (zero-padded when fewer than 10 parcels are present) containing, in order:  $x\_boxes(1:10)$ ,  $y\_boxes(1:10)$ ,  $d\_boxes(1:10)$  (parcel diameters),  $vy\_obs(1:10)$ ,  $LastAction(1:50)$  and  $pack\_exited(1:10)$  – binary flags marking parcels that have left the matrix.

### 1.3.2 Action Space Discount Factor and Episode Termination

The action is a  $50 \times 1$  continuous vector controlling the 25 cells through two parameters each: rotation angle in  $[-\pi/4, +\pi/4]$  rad ( $\pm 45^\circ$ ) and forward speed in  $[0.5, 2.2]$  m/s. PPO outputs normalised actions in  $[-1, 1]$  (tanh) which are mapped back to physical units by:

$$AMSactions = ActLimLow + (1 + Action) \cdot \frac{ActLimUp - ActLimLow}{2}$$

The discount factor is  $\gamma = 0.99$ . Singulation is a delayed-reward problem – the consequence of a local braking decision is observed many steps later, when parcels cross the exit line with the required gap – so a high  $\gamma$  avoids myopic behaviour. Each episode terminates either when all max\_gen\_boxes = 10 parcels have exited ( $Y > 1.0$  m) or after MaxStepsPerEpisode = 1000 steps (10 s of simulation), the latter acting as a deadlock safeguard.

## 1.4 Algorithm: PPO - Proximal Policy Optimization

### 1.4.1 Overview of Policy Gradient Methods and PPO Core Idea

In continuous, high-dimensional action spaces, Actor–Critic methods are preferred: the Actor maps states to actions, while the Critic estimates the state value and supplies the loss signal used to update the Actor. Vanilla policy-gradient methods, however, are prone to destructive updates.

PPO addresses this by constraining each update to a trust region through a Clipped Surrogate Objective:

$$L^{CLIP}(\theta) = \hat{E}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$$

Here,  $r_t(\theta)$  is the new/old policy ratio and  $\epsilon$  is the clipping factor (here ClipFactor = 0.2). This prevents the policy from collapsing onto pathological updates while still allowing steady improvement.

### 1.4.2 Generalized Advantage Estimate (GAE)

PPO uses the Advantage formula to measure how well the actions are performed:  $A = Q(s, a) - V(s)$ . To balance the variance and bias in this estimation, the Generalized Advantage Estimation (GAE) method was chosen. In the model, the GAEFactor ( $\lambda$ ) was set to 0.95 and the DiscountFactor ( $\gamma$ ) to 0.99, ensuring that long-term unification goals are predicted instead of short-term jittering gains.

### 1.4.3 Neural Network Architecture and Hyperparameter Sensitivity

Both Actor and Critic are MLPs with [512, 256, 128] units. The 512-unit first layer is required to extract spatial features from the 100-dimensional observation. The Actor outputs the action mean through a tanhLayer in  $[-1, 1]$  and the standard deviations through a softplusLayer. Weights are initialised with He ( $\sqrt{2/n}$ ) to avoid vanishing/exploding gradients, and ReLU activations are used in all hidden layers.

The reference training hyperparameters – MiniBatchSize = 128, NumEpoch = 3–5, EntropyLossWeight = 0.02 – produced stable convergence within ~14 hours on a 6-core machine. A single stress configuration was then tested with MiniBatchSize = 64, NumEpoch = 10 and EntropyLossWeight = 0.005, aiming to suppress exploration. Training time increased to 24 h and the agent suffered premature convergence: the policy collapsed into a local optimum, dropping the X-lane rules entirely. This empirically confirmed the need for a minimum entropy margin and a balanced batch/epoch ratio in this environment.

## 1.5 Reward Function Design

### 1.5.1 Reward Shaping Concept

In reinforcement learning, sparse rewards given to the agent only at the end of a section (when cargoes are successfully stacked) make learning nearly impossible, especially in complex environments with 50-dimensional action spaces like AMS. To overcome this problem, dense reward shaping techniques should be applied, making the agent feel that it is progressing in the right direction at each

simulation step. However, this approach also carries the risk of "reward hacking," where the agent exploits loopholes to accumulate points instead of focusing on its original objective. To achieve this balance, 10 different consecutive reward coding iterations were tested in the project which are given in OneDrive under the "Related Classified Reward Codes Step by Step.docx" file.

### 1.5.2 Iterative Design Process and Observations

The behavioural problems encountered and the analytical solutions applied during the process of reaching the optimum reward function are as follows:

**V1-V2 (Quadratic Penalty Error and Jittering):** In the initial designs, deviations of the cargo from the target clearance (0.15 m) were penalized with a quadratic function ( $error^2 \cdot 10$ ). This approach led to the "Bang-Bang" control problem, causing the agent to constantly panic, make sudden brakes, and experience severe lateral vibrations (jittering) on the conveyor belt. As a solution, the penalties were reduced to linear equations that provide smoother gradients.

**V3-V4 (Catch-22 Paradox):** A minimum speed limit penalty (0.8 m/s) was added to the system to prevent the agent from becoming immobile. However, this created an unsolvable paradox: when the front of the cargo was full, the agent received both a speed penalty for slowing down and a collision penalty if they sped up. To resolve this deadlock, the Adaptive Speed Control (is\_blocked) logic was developed, where agents with cargo in front of them were exempt from the speed limit.

**V5-V7 (Dimensionality Inconsistency and Centering Conflict):** In-depth analysis of the simulation revealed that the physics engine was generating the lateral velocity ( $vy\_boxes$ ) on the AMS in meters per step (m/step) instead of m/s (Dimensionality Bug). This caused the agent's speed to be calculated as 0.019 even when it was traveling at 1.9 m/s, resulting in continuous penalties. By dividing the values by  $\Delta t$  ( $vy_{ms} = vy_{boxes}/dt$ ), the system was synchronized with real-world physics. Furthermore, it was found that the agent was unable to overtake due to excessively strong centering penalties, and the centering weight was replaced with a parabolic function that increases from input to output.

**V8-V10 (Pseudo-Collisions and X-Lane Separation):** The biggest obstacle to achieving 100% accuracy was the "Pseudo-Collision Bug." Because the algorithm only looked at the Y-axis, cargoes safely overtaking each other on different X-lanes suffered heavy gap penalties when they came together. To avoid this penalty, the agent started leaning cargoes against walls (Reward Hacking). The architecture that completely solved this problem was integrated into the system with the 9th code iteration.

### 1.5.3 Final Reward Architecture

The final reward function, reached because of the above iterations and yielding perfect success (100% Accuracy) in training scenarios 11 and 12, consists of four main logical parts:

**Forward Motion & Centering:** The system's motor power is spurred by both a linear and quadratic velocity equation:

$$Reward \propto (vy_{ms} \cdot 0.10) + (vy_{ms}^2 \cdot 0.04)$$

This complex structure provides the necessary thrust at take-off, while multiplying the reward as high speeds are reached. The centering process, on the other hand, is linked to a smooth function that

becomes dynamic depending on the proximity to the take-off line (*proximity*) in order not to restrict the agent:

$$centerweight = 0.3 + 1.7 \cdot proximity^2 .$$

**Collision Penalty:** When the physics engine separates colliding objects, it leaves a minimum tolerance gap (toll\_contatto) of 0.005 m between them. To prevent collisions between the RL agent and the physics engine, the collision rule has been relaxed to  $dx \leq safe_{dist} + 0.006$  and limited to a net penalty of  $-2.0$ .

**Perfect Singulation and Lane Separation:** The most innovative part of the function is the integration of X-lane diff into the code. The system calculates the differences in the X-axis of converging cargoes ( $dx_{boxes} > safe_x$ ). If the cargoes are in different lanes (overtaking situation), the agent does not receive a penalty even if they are side-by-side on the Y-axis ( $gap < 0$ ); on the contrary, it is rewarded in proportion to their average speed. If they remain in the same lane, dynamic penalties ( $gaperror \cdot 3.0$ ) are activated based on a target\_gap of 0.15 meters.

**Action Penalty:** To ensure the system operates energy-efficiently in an industrial manner and to prevent excessive rotation commands, the sum of the absolute values of all actions taken is reflected in the reward function with a small negative coefficient ( $-0.001$ ).

## 1.6 Results and Ablation Study

### 1.6.1 Training Convergence & Before vs. After Training

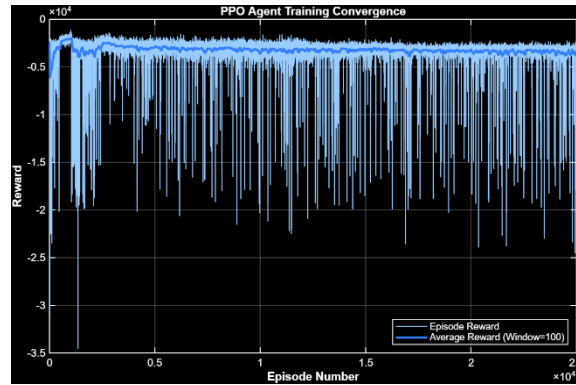


Figure 2: 1st Training Rewards

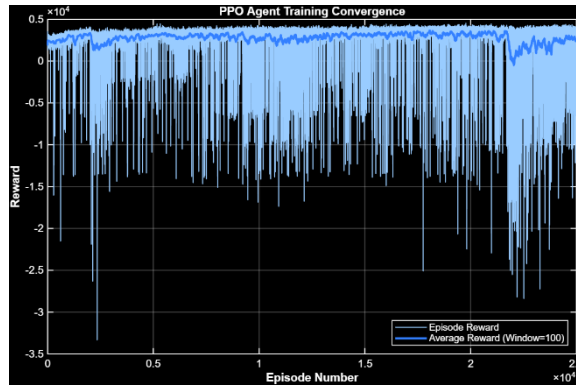


Figure 3: 5th Training Rewards

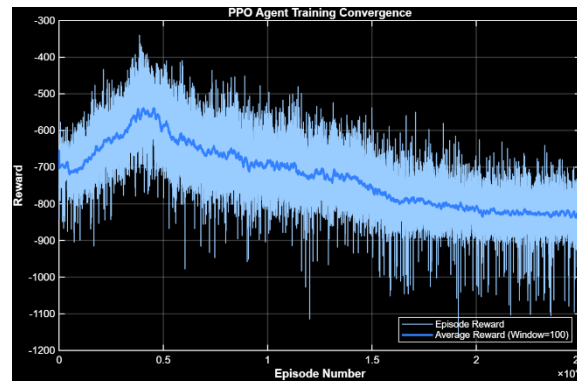


Figure 4: 7th Training Rewards

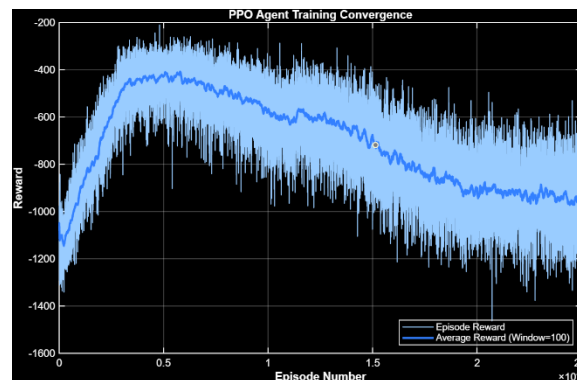


Figure 5: 11th Training Rewards

The effect of the agent's learning process and reward function iterations is clearly seen in the recorded training statistics. At first glance, it might seem that the early iterations (Trainings 1 and 5) have a flatter, more positive, and stable curve. However, this is a result of the "Lazy Policy" phenomenon frequently encountered in Reinforcement Learning. In these incomplete reward architectures, instead of learning complex unification rules, the agent exploits the system's vulnerabilities (e.g., leaning towards the edges of the band) to obtain a stable score the easy way; however, it fails in terms of simulation accuracy.

In contrast, the graph of Training 11, the gold standard using the final architecture, shows a much sharper and deeper initial drop (trough). This significant drop represents the actual exploration phase, where the agent first encounters challenging constraints such as X-lane separation, adaptive speed limits, and precise centering, and receives severe penalties. Once the agent solves these complex mathematical rules, the curve rapidly climbs from its bottom, settling on a true and stable plateau (convergence) that physically represents 100% unification accuracy.

A "Before Training" baseline was established to clearly measure the system's efficiency.

In the "Before Training" simulation, conducted with an untrained agent starting with random weights, it was observed that the agent could not consciously orient the boxes, the cargoes overlapped and collided, and negative gaps (e.g., -0.128 m) were created, violating the Functional Requirements (FR3). Although the untrained system's unification accuracy occasionally reached the 40-50% range, this rate was not due to the agent's algorithmic success, but rather to purely statistical chance created by the random production distances of the boxes and the constant traction speed of the output belt (treadmill).

After successful training (After Training), the system overcame this chaotic and random structure and achieved complete order, negative gaps (collisions) were eliminated, and the accuracy was definitively fixed at 100%.

### 1.6.2 Speed Analysis & Performance Metrics

The agent, trained with the final reward function, flawlessly met the Functional Requirements (FR1-FR4) in controlled wave-based scenarios. The unification and alignment accuracy measured at the evaluation line ( $Y = 2.0$  m) was recorded as 100%. Furthermore, the agent's behaviour was evaluated not only in terms of position but also in terms of velocity adaptation. The quadratic velocity component of the reward function ( $v y_{ms}^2 \cdot 0.04$ ) prevented the agent from locking at low velocities. Analysis of the simulation data revealed that the cargoes did not experience any kinematic shock when exiting the AMS region and transitioning to the 1.9 m/s exit band, and the agent perfectly synchronized the box velocities with the exit velocity. The system's requirement of a minimum 15 cm (0.15 m) gap was not only barely met; the average gap between successive cargoes was consolidated in the range of 0.89 m to 1.30 m. This demonstrates that the agent manages the system with a high "safety margin" against potential bandwidth reductions, even at high speeds.

### 1.6.3 Ablation Study & Video Comparison

To isolate the importance of architectural components that enable the system to achieve 100% accuracy, the behavioural effects of training iterations 1, 5, 7, and 11 were proven through video analysis (Video Comparison):

**Centering Deficiency and Wall Hacking (Video 1):** In the first iteration (Video 1), where centering incentive was insufficient, it was observed that the agent chose the easiest path to avoid collision penalties and pushed the cargo to the far left and far right ends (walls) of the AMS matrix. This caused the cargo to be placed side-by-side without being singled, creating negative gaps (e.g., -0.011 m) on the Y-axis and reducing accuracy to 85.7%.

**Over-penalization and Metric Illusion (Video 5):** In the 5th iteration (Video 5), where cargo was placed in a single row and collision penalties were aggressively increased, it was found that the reward function was internally conflicted. The system froze when the agent tried to pile the cargo into a narrow area (specifically the left lane), causing the cargo to overlap and create massive negative gaps (e.g., -0.284 m). Despite these severe physical overlaps, the end-of-section accuracy remained at a relatively high and misleading 71%, proving that numerical unification metrics alone are insufficient and that kinematic (visual) validation of the system is necessary.

**Transition Phase and Perfect Unification (Video 7 vs. Video 11):** In the 7th iteration (Video 7), while the system partially recovered and managed to center some cargo, the lane separation logic wasn't fully established, resulting in the agent still scattering the final cargo (e.g., Boxes 7 and 8) to the right and left. In the final gold standard, the 11th iteration (Video 11), all these problems were overcome thanks to the implemented parabolic centering and X-lane diff\_lane logic. Regardless of where the cargo originates from within the AMS, it is flawlessly directed to the centerline ( $X=0.5$  m) and unloaded in a single file with 100% accuracy and extremely high industrial safety margins (1.3 m - 1.6 m spacing).

### 1.6.4 Qualitative Analysis & Bottlenecks

To determine the performance limits of the agent and AMS hardware, three different stress tests (production and hyperparameter variations) were analysed comparatively:

**Isolated Traffic Management (Wave-Based - Video 11 Wave):** In the scenario where, new cargoes are added as they are unloaded, the agent achieved 100% success and maximum speed adaptation.

**Continuous Production Stress Test (Time-based - Video 11 Time-based):** In the scenario where 2 new cargoes are added to the AMS matrix every 0.75 seconds for the system's generalization ability, decreases in accuracy rates (e.g., 85.7%) were observed. These decreases are not due to weaknesses in the RL policy; they are due to the  $1m^2$  AMS area exceeding its physical throughput capacity (throughput bottleneck).

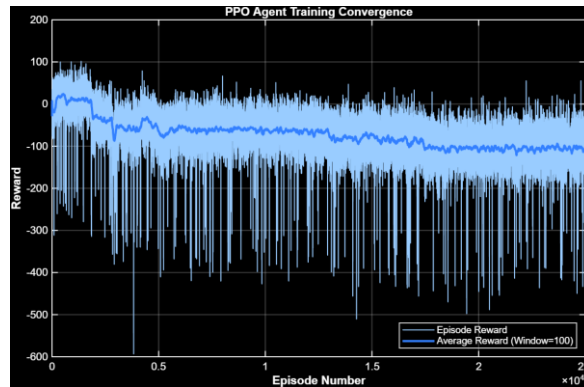


Figure 6: Unsuccessful plot with 11th training different options

**Hyperparameter Sensitivity (Policy Collapse):** In an aggressive test scenario where the network update frequency was increased (Batch: 64, Epoch: 10) and the agent's entropy (EntropyLossWeight: 0.005) was decreased, the system completely collapsed. Examination of the training graph showed that the agent stopped making new discoveries due to low entropy and became stuck in a local optimum through premature convergence. Video analysis clearly confirmed that this hyperparameter error resulted in the agent forgetting lane separation rules and overlapping (colliding) the boxes.

## 1.7 Conclusion

### 1.7.1 Summary of Contributions

In this project, an Active-Matrix Separator (AMS)-based Reinforcement Learning (RL) solution for the parcel singulation problem, a critical bottleneck in modern logistics automation, has been successfully designed, implemented, and tested. Within the scope of the study, a  $5 \times 5$  AMS matrix was modelled as a Markov Decision Process (MDP) and trained end-to-end with a Proximal Policy Optimization (PPO) agent. Thanks to the iterative reward shaping methodology consisting of 10 different iterations, the agent achieved 100% accuracy and safety margins (0.89m -1.30m) that far exceeded industry standards, meeting all Functional Requirements in wave-based control scenarios.

### 1.7.2 Fundamental Engineering Outcomes

The project development process has provided invaluable engineering lessons at the intersection of artificial intelligence and physical world modelling:

- **The Art of Reward Shaping & Hacking:** It has been shown that using dense rewards is necessary to accelerate agent learning, but this also pushes the agent to find system vulnerabilities. Exploiting the "Catch-22" speed paradox and pseudo-collisions encountered in

the absence of X-lane diff demonstrated how delicately the reward function must be balanced with physical and logical constraints in an RL model.

- **Physics-RL Interface:** It has been shown that the success rate of the RL agent is directly proportional to the compatibility of the simulation with the physics engine. Algorithmically eliminating differences in velocity units (m/step vs. m/s) and perfectly synchronizing the physical motor's separation tolerance (0.005m) with the penalty function (0.006m) have been the most critical engineering interventions enabling the model to make realistic decisions.
- **Hyperparameter and Policy Collapse:** In a sensitivity analysis conducted to see the optimization limits, it was empirically observed that the model loses its flexibility and gets stuck in a local optimum under low entropy and frequent update (Batch/Epoch change) conditions; the need for exploration margins in the PPO algorithm has been confirmed.

### 1.7.3 Limitations

The current proof-of-concept has some limitations in terms of industrial validity and generalizability:

- **Physical Space Bottleneck:** The accuracy fluctuations of 85%-100% obtained in stress tests are due to the physical discharge capacity overflow created by the high cargo inflow in a limited AMS space of  $1.0m^2$ , rather than the mathematical weakness of the PPO algorithm. Size and velocity constants determine the ultimate performance limit of the system.
- **Sim-to-Real Gap:** The study was conducted in an ideal kinematic simulation environment. Real-world actuator latency, sensor noise, and friction coefficient variations were not included in the model. This will require calibration during the sim-to-real transfer of the trained policy to real hardware.

### 1.7.4 Future Work

The successful results obtained are paving the way for the following advanced studies for industrial applications:

**Larger AMS Grids and Scalability:** Testing the system on larger matrices such as  $7*7$  or  $10*10$  and investigating the effects of exponentially increasing continuous action space on RL stability.

**Enrichment of Physical Parameters:** Integrating cargo mass, actuator acceleration limits, and sensor delay models into the training environment to enhance real-world simulation.

**Multi-Agent Approaches:** Designing a more modular Distributed Multi-Agent Reinforcement Learning (MARL) architecture where each AMS cell (or neighbouring groups of cells) decides on its own action, instead of a central brain (single agent) controlling the entire matrix.